Consistenţa şi replicarea datelor în Internet

Bazat pe "Distributed Systems" de AS Tanenbaum

Motive pentru replicare

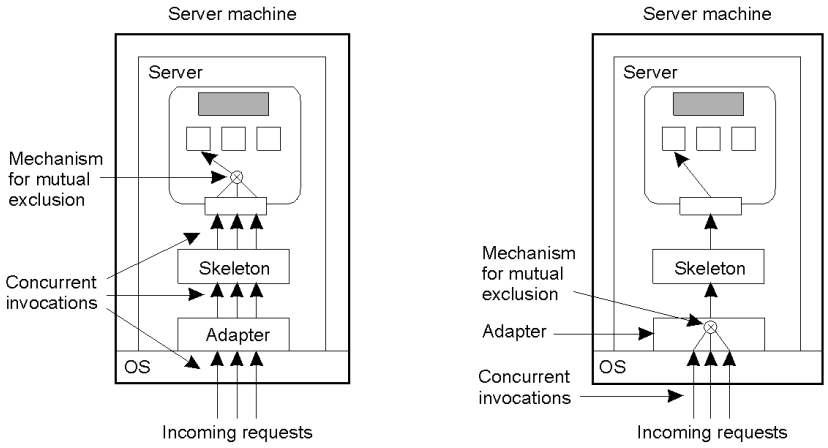
Principalele doua motive pentru replicare sunt cresterea fiabilitatii si cresterea performantei. Prin utilizarea replicarii, sistemul continua sa lucreze chiar dupa caderea unei replici. In cazul unor atacuri ce afecteaza unele replici, prezenta unor replici corecte sta la baza producerii unor rezultate corecte.

In ce priveste performanta, sunt mai multe aspecte. Mai intai, atunci cand mai multe procese folosesc o aceeasi colectie de date, replicarea acesteia permite ca fiecare copie sa serveasca un numai mai mic de procese, de aici rezultand o scadere a timpului de raspuns. Aceasta se poate realiza si prin distribuirea geografice a replicilor, caz in care clientii pot accesa cea mai apropiata resursa.

Replicarea ridica probleme de consistenta. Asa cum am vazut intr-un curs precedent, pastrarea unor resurse web in cache mareste performanta dar cere masuri speciale pentru a oferi, mereu, cea mai actuala copie a paginii.

Controlul accesului concurent la un obiect

Replicarea reclama utilizarea unor mecanisme speciale in accesarea replicilor. Exemplificam aceasta idee pentru sisteme de obiecte distribuite si sisteme de fisiere distribuite. Prezentam mai intai mecanismele de acces concurent la un obiect.

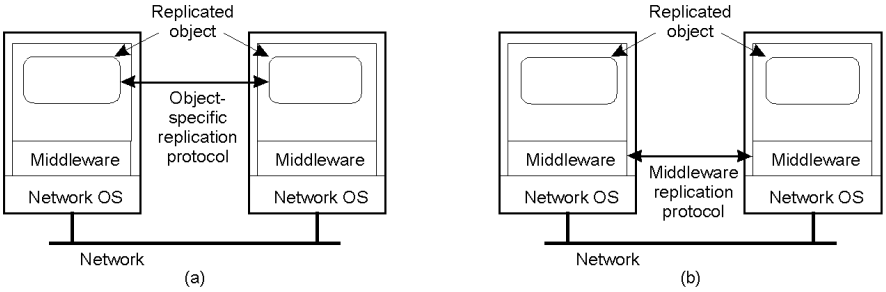


Manipularea accesului concurent poate fi asumata de catre obiect (ca in figura a) sau de un adaptor extern obiectului (ca in figura b). In primul caz, mecanismul de excludere mutuala este inclus in obiect si este concretizat prin primitive de sincronizare folosite explicit de programatorul care dezvolta metodele obiectului.

In al doilea caz, adaptorul implementeaza politica de activare a obiectului. El poate permite invocari concurente (figura a) sau un singur thread per obiect (figura b). In solutia a, protectia trebuie facuta la nivelul obiectului, asa cum am aratat anterior. De exemplu, in Java metodele se declara *synchronized*. In solutia b, protectia se face la nivelul serverului.

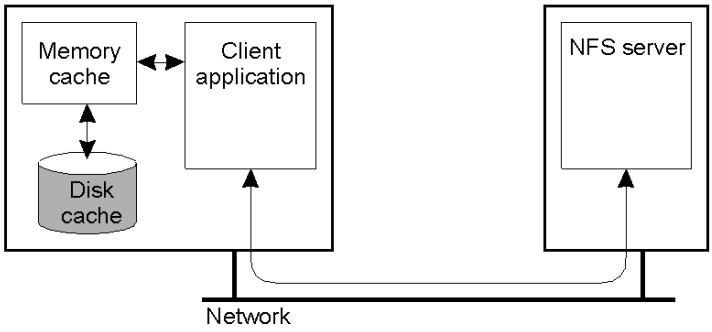
Invocari concurente ale obiectelor replicate

In invocarea unui obiect replicat, aceleasi modificari de stare trebuie sa apara in toate replicile sale. O solutie este ca invocarile sa se faca in aceeasi ordine la toate replicile. Sunt doua implementari posibile: obiectele sunt "constiente" de replicare (ca in Globe) si folosesc un protocol specific de replicare a operatiilor realizate asupra obiectelor. In al doilea caz, sistemul distribuit (prin middleware) face gestiunea replicilor (Piranha), de exemplu ordonarea totala a invocarilor la toate replicile. Revenim mai tarziu, cu detalii referitoare la a doua solutie.



Replicare in sistemele de fisiere

In cel de al doilea exemplu, consideram problema eficientizarii accesului partajat la fisiere. Pentru NSFv4, replicarea la client se face prin utilizarea unei memorii cache si, eventual, a unei extensii cache pe disc. Se realizeaza caching pentru date, atribute, handlere, si directoare. Replicarea fisierelor se poate face si la server sau in sisteme p2p, lucru discutat mai tarziu in acest curs.



Modele de consistenta

In principiu, **consistenta** inseamna ca un proces obtine acceasi valoare a unei resurse indiferent de replica accesata. **Consistenta stricta** inseamna ca aceasta valoare corespunde celei mai recente actualizari a resursei. Ea este greu de pastrat, motiv pentru care aplicatiile recurg la acceptarea unor constrangeri mai slabe de consistenta. Acestea depind de tiparele de acces si de actualizare a resurselor, dar si de cerintele impuse de aplicatii.

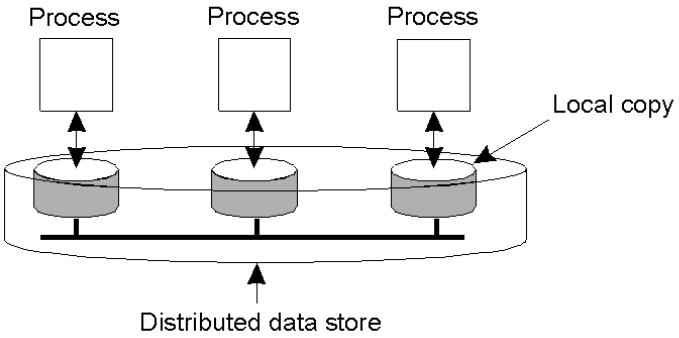
Modele de consistenta se impart in doua categorii:

- modele centrate pe date, care corespund executiei operatiilor de citire si scriere asupra unor date disponibile in memorii distribuite partajate, baze de date distribuite partajate, sisteme de fisiere distribuite partajate, numite generic depozite de date;

- modele centrate pe client, care garanteaza consistenta datelor accesate din diferite locatii de un acelasi client, la diferite momente de timp.

Modele de consistenţă bazate pe date

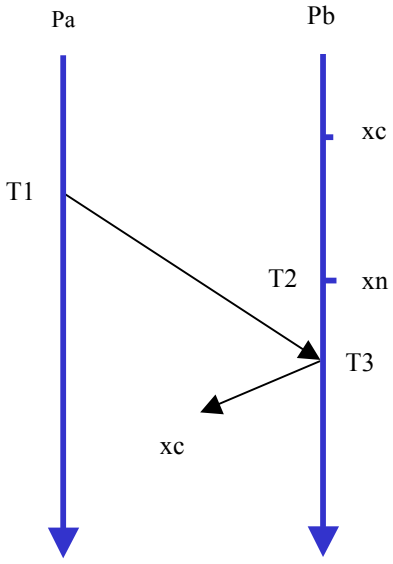
Organizarea generala a unui depozit de date distribuit si replicat este prezentata in figura urmatoare. Depozitul de date cuprinde mai multe copii, fiecare proces, din cele care partajeaza datele, avand acces direct la una din copii numita copie locala (procesului). Un proces poate realiza una din operatiile read si write. Operatia **read** se executa pe copia sa locala si un modifica starea datelor din depozit. Operatia **write** se executa pe copia locala procesului executant, dar modificarea este propagata la celelalte copii. Timpii de propagare difera de la o copie la alta, astfel ca modificarile facute asupra unei copii locale unui proces au efecte la momente de timp diferite asupra celorlalte replici si sunt percepute diferit de celelalte procese care citesc din copiile lor locale.



Un model de consistenta este un contract intre proces si data store, care fixeaza regulile de functionare. In mod normal, o operatie **read** ar trebui sa intoarca rezultatul ultimei operatii de modificare a unei date, indiferent de copia locala accesata. In absenta unui ceas global, este greu de spus care este ultima modificare in raport cu operatia de citire. Din acest motiv, se recurge la alte reguli, care reclama mecanisme mai simple de aplicare.

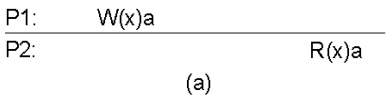
Consistenta stricta

Conform acestui model, orice citire a unei date x intoarce valoarea rezultata din cea mai recenta scriere a lui x. Pentru a intelege mai bine, sa consideram exemplul a doua masini A si B si a doua procese, Pa pe masina A si Pb pe masina B. Variabila **x** este memorata doar pe B si valoarea curenta este **xc**.
La momentul T1, Pa citeste **x** (trimite un mesaj lui B sa citeasca x). La T2>T1, Pb scrie **x**; noua valoare este **xn**. La T3>T2, mesajul de citire de la Pa soseste la masina B. Consistenta stricta impune ca B sa intoarca valoarea **xc**, care reprezinta valoarea variabilei **x** la momentul in care Pa a facut cererea de citire. Deci, valoarea intoarsa de Pb trebuie sa fie diferita de **xn**, cea din momentul P3 la care Pb alcatuieste mesajul de raspuns catre Pa.

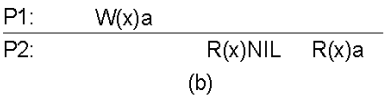


Consistenţa Strictă (2)

In modelul de consistenta stricta, se pastreaza ordinea absoluta globala in timp. Toate scrierile sunt instantaneu vizibile tuturor proceselor.



(a)



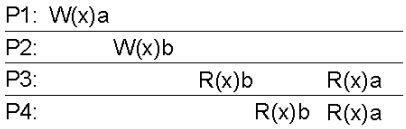
(b)

In figura se prezinta comportarea a doua procese operand pe acelasi item, in doua situatii:
a) Memoria este strict consistenta.
b) Memoria nu este strict consistenta.

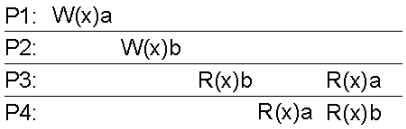
Consistenta stricta este greu de implementat deoarece presupune pastrarea unui timp global si o gestiune a valorilor datelor si a momentelor de timp la care s-au facut modificari asupra lor. Programele concurente nu sunt bazate pe timpul global sau pe viteza proceselor (de ex. Producer / Consumer). Aceasta justifica necesitatea unor modele mai slabe.

Consistenta Secvenţială (1)

Consistenta secventiala se enunta astfel: **Rezultatul oricarei executii este acelasi cu cel obtinut daca operatiile (read, write) tuturor proceselor asupra depozitului de date sunt executate intr-o secventa oarecare si operatiile fiecarui proces individual apar in aceasta secventa in ordinea specificata de programul sau.**
Orice intretesere valida a operatiilor read si write este acceptabila atat timp cat toate procesele vad **aceeasi** intretesere de operatii. Operatiile nu au amprente de timp.



(a)



(b)

In figura sunt prezentate: (a) un depozit de date secvential consistent si (b) unul care nu este secvential consistent. Situatia din (b) nu este consistenta deoarece procesele P3 si P4 nu "vad" aceeasi intretesere de operatii: P3 vede mai intai operatia lui P2 (care da lui x valoarea b) apoi a lui P1, in timp ce P4 vede mai intai operatia lui P1 si apoi a lui P2.

Consistenţa Secvenţială (2)

Fie x, y, si z care sunt initializate la 0. Consideram urmatoarele trei procese concurente:

Proces P1	**Proces P2**	**Proces P3**
x = 1;	y = 1;	z = 1;
print (y, z);	print (x, z);	print (x, y);

O operatie de atribuire corespunde unui write; un print corespunde la doua operatii read simultane.
Sunt posibile 90 de ordonari valide, diferite ale instructiunilor. Numarul se calculeaza astfel: sunt in total 720 (=6!) permutari posibile a 6 instructiuni, din care jumatate trebuie eliminate pentru ca un respecta ordinea operatiilor din primul proces; din ce ramane, jumatate trebuie eliminate pentru ca un respecta ordinea operatiilor din al doilea proces; la fel si pentru al treilea. Raman 90 (=720/8) de ordonari valide. Oricare dintre aceste ordonari trebuie sa fie acceptate de procesele care se supun modelului de consistenta secventiala, cu observatia ca oricare doua procese vad aceeasi ordonare.
Cu toate ca sunt 90 de ordonari valide, sunt mai putin de 64 tipare de signatura, unde o signatura este concatenarea iesirilor lui P1, P2 si P3 in aceasta ordine. Numarul de 64 corespunde tuturor combinatiilor de 6 biti posibile cu cele trei grupuri de 2 biti tipariti de fiecare proces.

Consistenţa Secvenţială (3)

Prezentam mai jos patru secvente de executie valide pentru procesele anterioare. (Axa verticala este timpul) si semnaturile corespunzatoare. De observat ca "Prints" difera de semnaturi, deoarece ele specifica secventa de cate doua cifre binare printate de procese in ordinea in care au loc operatiile de tiparire. De exemplu, in cazul (c) procesele tiparesc in ordinea P3, P2, P1. Evident, sunt si cazuri in care iesirea programului corespunde cu semnatura.

x = 1;
print (y, z);
y = 1;
print (x, z);
z = 1;
print (x, y);
 	
Prints: 001011
 	
Signatura:
001011
(a)
	x = 1;
	y = 1;
	print (x,z);
	print(y, z);
	z = 1;
	print (x, y);
	
	Prints: 101011
	
	Signatura:
	101011
	(b)
		y = 1;
		z = 1;
		print (x, y);
		print (x, z);
		x = 1;
		print (y, z);
		
		Prints: 010111
		
		Signatura:
		110101
		(c)
			y = 1;
			x = 1;
			z = 1;
			print (x, z);
			print (y, z);
			print (x, y);
			
			Prints: 111111
			
			Signatura:
			111111
			(d)

Nu toate cele 64 semnaturi sunt posibile. De exemplu, semnaturile urmatoare nu pot fi obtinute:
000000 banal, ar insemna ca procesele tiparesc inainte de modificarea vreunei variabile ceeace este imposibil.
001001 Mai dificil de analizat. In principiu,

Primii doi 00 implica print(y,z) executat inainte ca y si z sa fie modificate, deci P1 executa ambele instructiuni inainte de celelalte procese; ca rumare x devine 1
10 implica P2 executa ambele instructiuni inainte de P3;
01 implica P3 executa ambele instructiuni inainte ca p1 sa schimbe x in 1 ceeace contrazice prima afirmatie ca P1 se executa primul.

Consistenţa Cauzală (1)

Unele operatii sunt **legate cauzal**. De exemplu, daca procesul p executa operatia (1): write x; apoi procesul q executa (2): read x; si (3) write y, operatiile (1) si (3) sunt legate cauzal deoarece rezultatul operatiei (3) depinde (in general) de operatia (1) al carei efect este resimtit de procesul q prin operatia de citire (2) care precede scrierea (3).
Doua operatii care nu sunt dependente cauzal sunt **concurente**.

Conditia necesara stipulata in modelul de consistenta cauzala este ca: **Operatiile "write" care sunt potential legate cauzal trebuie sa fie vazute de toate procesele in aceeasi ordine. Scrierile concurente pot fi vazute in diferite ordini pe masini diferite**.

Consistenţa Cauzală (2)

P1:	W(x)a		W(x)c		
P2:		R(x)a	W(x)b		
P3:		R(x)a		R(x)c	R(x)b
P4:		R(x)a		R(x)b	R(x)c

In exemplul de mai sus, $W_1(x)a$ si $W_2(x)b$ sunt dependente cauzal in timp ce $W_2(x)b$ si $W_1(x)c$ sunt concurente. Aceasta secventa este permisa cu o memorie cauzal-consistenta, dar nu cu una secvential sau strict consistenta.

Consistenta Cauzala (3)

Violarea consistentei cauzale este prezenta in urmatorul exemplu. (Cele doua scrieri sunt legate cauzal dar sunt vazute in ordini diferite de P3 si P4.)

P1: W(x)a					
P2:		R(x)a	W(x)b		
P3:				R(x)b	R(x)a
P4:				R(x)a	R(x)b

(a)

O secventa corecta de evenimente intr-o memorie cauzal-consistenta este data in urmatoarea figura. De data aceasta, cele doua scrieri nu mai sunt legate cauzal.

P1: W(x)a					
P2:			W(x)b		
P3:				R(x)b	R(x)a
P4:				R(x)a	R(x)b

(b)

Implementarea consistentei cauzale presupune pastrarea unui graf de dependente cauzale si se poate baza pe ordonarea evenimentelor folosind vectori de timp.

Consistenta FIFO

In modelul de consistenta FIFO trebuie respectata urmatoarea conditie:
Scrierile facute de un singur proces sunt vazute de toate celelalte procese in ordinea in care au fost executate, dar scrierile din procese diferite pot fi vazute in ordini diferite de procese diferite.
Consistenta FIFO este usor de implementat etichetand fiecare operatie write cu perechea (proces, numar secventa) si facand operatiile write ale fiecarui proces in ordinea numerelor de secventa.
O secventa valida de evenimente pentru consistenta FIFO este urmatoarea:

P1: W(x)a						
P2:	R(x)a	W(x)b	W(x)c			
P3:				R(x)b	R(x)a	R(x)c
P4:				R(x)a	R(x)b	R(x)c

Fie doua procese concurente:

Process P1	**Process P2**
x = 1;	y = 1;
if (y = = 0) kill (P2);	if (x = = 0) kill (P1);

Initial x si y sunt 0. P1 scrie in x si citeste y inainte sa vada scrierea lui P2 in y. P2 scrie in y si citeste x inainte sa vada scrierea lui P1 in x.
Conform consistentei FIFO, ambele procese pot fi omorate deoarece scrierile pot fi vazute de procese in ordini diferite fiind astfel posibil ca in ambele procese decizia sa fie evaluata pozitiv.
In consistenta secventiala exista sase posibile intreteseri, nici una cu ambele procese omorate.

Gruparea operatiilor

Datele partajate pot fi asociate cu o variabila de sincronizare, care permite gruparea operatiilor executate asupra lor si "forteaza" respectarea ordinii operatiilor din grup.
Un proces foloseste **acquire** si **release** pe variabilele de sincronizare si anume:

- acquire – cand intra in sectiunea critica si datele protejate de variabila de sincronizare sunt facute consistente
- release - cand iese din sectiunea critica.

Un model specific de folosire a variabilelor de sincronizare este urmatorul:

- Fiecare variabila de sincronizare are un proprietar curent.
- Proprietarul poate modifica datele protejate de variabila de sincronizare, in mai multe sectiuni critice (ramane proprietar pe durata acestor operatii).
- Un proces care vrea sa acapareze variabila (acquire) trebuie sa faca o cerere proprietarului, devenind noul proprietar.
- Exista si un mod ne-exclusiv in care mai multe procese detin variabila doar pentru citirea datelor protejate.

Consistenta la intrare - Criterii de consistenta

Un proces nu poate termina un acquire pe o variabila de sincronizare pana cand nu s-au actualizat toate datele partajate protejate de acea variabila.

Pentru ca un proces sa modifice date partajate, el trebuie sa capate accesul exclusiv la variabila de sincronizare care protejeaza acele date (nici un alt proces sa nu detina variabila de sincronizare, nici macar in mod ne-exclusiv)

Dupa un acces exclusiv la o variabila de sincronizare, orice acces ne-exclusiv ulterior al unui alt proces la acea variabila de sincronizare este admis doar dupa ce procesul a preluat de la proprietarul variabilei de sincronizare cele mai recente copii ale variabilelor partajate protejate.

Consistenta la Intrare

O secventa valida pentru consistenta la intrare este urmatoarea:

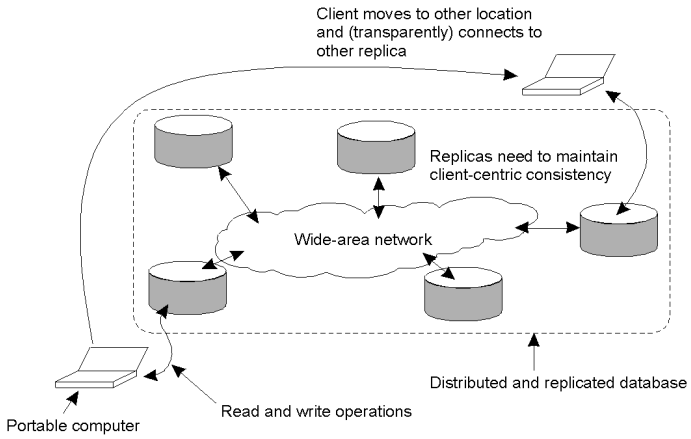
P1:	Acq(Lx)	W(x)a	Acq(Ly)	W(y)b	Rel(Lx)	Rel(Ly)			
P2:							Acq(Lx)	R(x)a	R(y)NIL
P3:								Acq(Ly)	R(y)b

Fiecare variabila partajata x este asociata cu o variabila de sincronizare (lock) Lx. In acest caz, P1 face un acquire pe x si scrie aceasta variabila, dupa care face acelasi lucru pentru y. Ulterior elibereaza ambele variabile de sincronizare. P2 citeste corect pe x (deoarece face acquire pe ea) dar nu si pe y. Ca urmare, valoarea citita de P2 nu este cea actualizata. P3 face un acquire pe y si o citire corecta a acestei variabile.

Modele de Consistenta Centrata pe Client

Consistenta centrata pe client este un caz special in care lipsesc actualizarile simultane. Un exemplu este o baza de date distribuita la care un client face un acces din locatia A pentru operatii de citire si scriere. Ulterior, facand acces din alta locatie, sa zicem B, clientul isi continua activitatea accesand o alta replica a bazei de date. Daca nu se iau masuri speciale de consistenta astfel ca replica din locaita B sa fie actualizata conform modificarilor facute in locatia A, clientul poate detecta inconsistente: modificarile facute in A nu au fost propagate, modificarile pe care le face in B pot intra in conflict cu cele din A etc. Alte exemple se pot gasi in utilizarea sistemelor DNS si Web.
In aceste exemple, consistenta poate fi realizata in final (eventual consistency) deoarece, in absenta indelungata a unor actualizari, toate replicile vor deveni treptat consistente. Acest "model" de consistenta este acceptabil cand clientul acceseaza mereu aceeasi replica.
In consistenta "Client-centric", un utilizator poate opera pe replici diferite. Ea da garantii unui singur client privind consistenta acceselor la replici diferite ale acelui client.

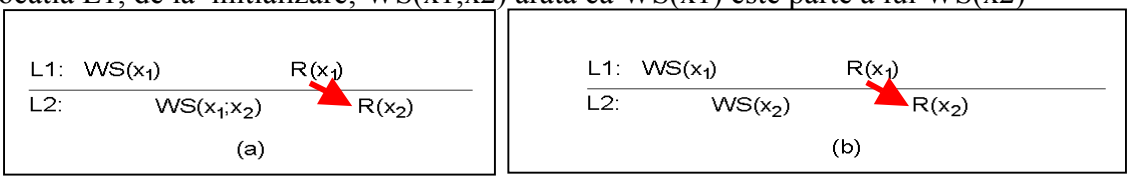
Consistenta finala vs. Client-centric



In figura este prezentat un utilizator mobil accesand diferite replici ale unei baze de date distribuite. Valorile citite si modificarile facute pe o replica trebuie sa fie regasite la cealalata replica.
Discutam in continuare cateva modele de consistenta centrate pe client.

Citiri monotone (Monotonic Reads)

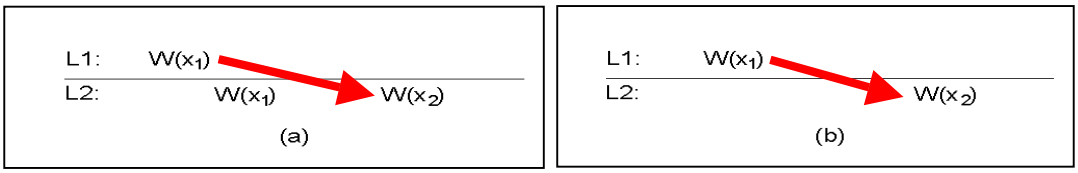
Daca un proces citeste valoarea unei date x, orice citire succesiva a lui x de catre acel proces va returna acea valoare sau o valoare mai noua.
Un exemplu este o baza de date distribuita pentru e-mail. La conectarea la un server e-mail diferit, acel server ofera cel putin informatia vazuta pe serverele vizitate anterior.
In cele ce urmeaza, notatia WS(x1) specifica seria de operatii de scriere pe x executate in locatia L1, de la initializare; WS(x1;x2) arata ca WS(x1) este parte a lui WS(x2)



In figura se arata operatiile *read* excutate de un singur proces *P* pe doua copii locale diferite ale aceleiasi baze de date. In (a) este prezentata o memorie consistenta "monotonic-reads". In (b) se arata o memorie ne-consistenta "monotonic-reads" deoarece operatiile de actualizare WS(x1) nu au fost propagate de la L1 la L2. Ca urmare, citirea de la L2, care este succesiva celei de la L1, nu va returna o valoare mai noua decat cea returnata de citirea de la L1.

Scrieri monotone (Monotonic Writes)

O operatie *write x* a unui proces este terminata inaintea oricarei operatii *write x* ulterioare a aceluiasi proces.
Exemplu: actualizarea unei biblioteci software facuta la un server S1. La actualizarea ulterioara a unui program la un server S2, trebuie sa se asigure ca toate componentele de care depind compilarea si linkeditarea programului, inclusiv biblioteca software au fost actualizate conform modificarilor de la serverul S1.
Obs. Seamana cu consistenta FIFO.



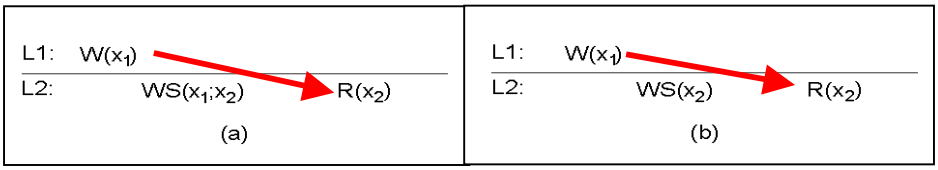
In figura sunt ilustrate operatiile de scriere executate de un proces *P* pe doua copii locale diferite ale aceluisi depozit de date. Sunt evidentiate un depozit consistent "monotonic-writes" (a) si un depozit ne-consistent "monotonic-writes" (b).

Citirea scrierilor proprii (Read Your Writes)

Efectul unei operatii *write x* a unui proces P va fi totdeauna vazut de operatiile *read x* executate ulterior de acelasi proces P.

Exemplu: actualizarea paginilor Web si observarea efectelor. Actualizarea trebuie sa asigure ca browserul arata cea mai noua versiune si nu vechea copie din cache.

Similar, la actualizarea unui password trebuie ca urmatoarea logare sa poata fi facuta cu noua parola.

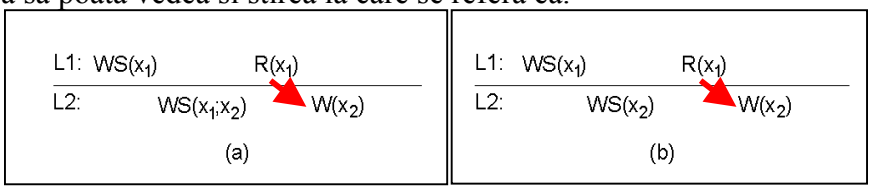


In figura se arata un depozit care ofera consistenta "read-your-writes" (a) si un depozit care nu ofera (b).

Scrierile urmeaza citirile (Writes Follow Reads)

O operatie *write x* executata de un proces P dupa o operatie prealabila *read x* a aceluiasi proces se executa garantat pe aceeasi valoare a lui *x* sau pe una mai recenta decat cea citita.

Exemplu: un utilizator citeste un articol A la care raspunde cu B; B va fi scris in orice copie a newsgroup-ului doar dupa ce A a fost scris de asemenea pentru ca cineva care vede replica sa poata vedea si stirea la care se refera ea.



In figura se prezinta un depozit care ofera consistenta "writes-follow-reads" (a) si un depozit care nu ofera (b).

Implementarea

Intr-o implementare "naiva", fiecarei operatii write i se asociaza un identificator global unic, de catre serverul care accepta operatia pentru prima data. Pentru fiecare client se pastreaza doua seturi de identificatori de write:

Read – identificatori ai operatiilor write relevante pentru operatiile read executate de client

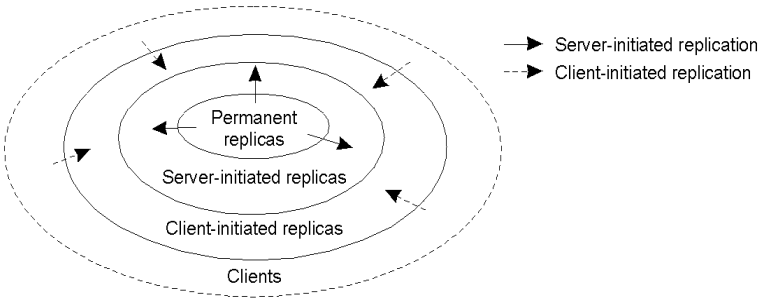
Write – identificatori ai operatiilor write executate de client.
La implementarea modelului "citiri monotone", pentru fiecare read:

Serverul primeste setul read al clientului
Verifica daca toate operatiile write au fost facute local
Daca nu -> contacteaza celelalte servere si face actualizarea
Serverul face citirea
Serverul actualizeaza setul read cu operatiile write realizate si care sunt relevante pentru client.

Actiuni similare sunt executate pentru celelalte modele

Protocoale de distributie - Plasarea Replicilor

Organizarea logica a diferitelor categorii de copii ale depozitelor de date in trei inele concentrice.



Sunt evidentiate urmatoarele:
Replici permanente. Acest caz se refera la servere "replica" situate in acelasi sistem (cluster) si la mirroring – replici distribuite geografic.

Replici initiate de server. Sunt create dinamic de servere pentru a imbunatati performanta sistemului prin includerea de replici apropiate de clienti si de replici pentru reducerea incarcarii unui server.
Una din problemele de implementare este "cand si unde" sa se plaseze replicile. Intr-o solutie posibila (Rabinovich), fiecare server tine evidenta numarului de accese per fisier si a originii cererilor de acces de la diversi clienti.

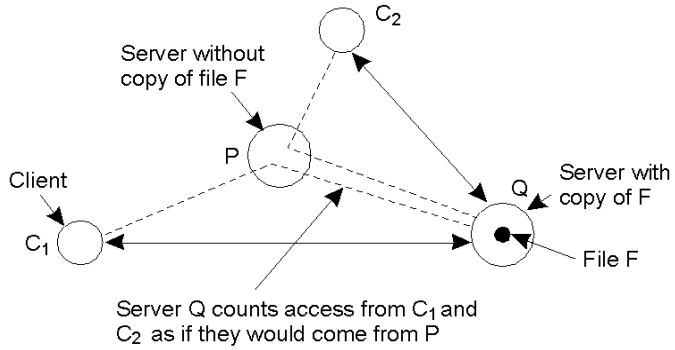


Figura prezinta cazul unui fisier F plasat pe serverul Q. Fisierul este accesat de clienti (C1, C2,...) prin intermediul serverului P care nu are o copie a lui F. Serverul Q contorizeaza accesele la fisierul F ca provenind de la P. Pentru ameliorarea performantelor, serverul Q poate decide mutarea lui F pe serverul P sau replicarea lui pe acest server. Mai precis, politica de replicare poate fi urmatoarea:
- daca numarul de cereri pentru F este mai mic decat un prag stergere del(Q,F) atunci Q sterge fisierul F;
- daca numarul de cereri pentru F este mai mare decat un prag de duplicare rep(Q,F) atunci F este replicat pe un alt server;
- daca numarul de cereri este intre cele doua praguri atunci Q poate decide mutarea lui F pe alt server (de exemplu, pe P).

Replicile initiate de clienti (caches) sunt gestionate de clienti folosind informatii de la servere si sunt utilizate doar pentru a imbunatati timpul de acces la date. In aceste replici, datele sunt pastrate pentru un timp limitat. Cache-urile pot fi plasate la clienti sau pot fi partajate si plasate in apropierea mai multor clienti dintr-o anumita zona.

Propagarea Actualizarilor

Exista trei posibilitati de propagare a actualizarilor:
- propagarea de la o replica la alta a datelor schimbate (propagarea starii)
- propagarea operatiilor de actualizare
- propagarea notificarilor de modificare.

Propagarea datelor (starii) este utila cand rata operatiilor read / write este ridicata si fiecare noua operatie de citire trebuie sa gaseasca imediat cele mai recente date. Pentru eficienta transportului, se recurge la propagarea log-urile modificarilor.
Propagare operatiilor de actualizare este numita si replicare activa. In acest caz operatiile de modificare se aplica pe fiecare replica. Varianta are sens atunci cand volumul codului este mai mic decat cel al datelor modificate astfel ca rezulta un cost comunicare redus. In plus, repetarea operatiilor la diverse replici nu trebuie sa incarce prea mult sistemul.
Notificarile sunt utilizate de protocoalele de invalidare prin care replicile sunt anuntate ce parti ale datelor nu mai sunt actuale. In cazul in care acele parti invalidate sunt accesate de procese, replicile vor cere mai intai actualizarea lor si apoi vor efectua operatiile cerute asupra datelor actualizate. Notificarile utilizeaza largimi de banda reduse.

Protocoale Pull versus Push

Protocoalele "Push" se aplica de regula replicilor permanente si "server initiated" care necesita un grad inalt de consistenta. Propagarea este initiata de serverul care a facut actualizarea, chiar fara ca aceasta operatie sa fie ceruta de celelalte replici.
Protocoalele "Pull" sunt folosite, de regula, pentru cache-uri client, dar nu numai. Propagarea este initiata de replica ne-actualizata care intreaba un server daca s-au facut modificari si efectueaza actualizarile in caz afirmativ. Sunt utile cand rata read / write este redusa.

Element	Push-based	Pull-based
Stare server	Lista replicilor si a cache-	Nimic

	urilor client	
Message transmise	update sau invalidare plus fetch update ulterior	poll si update
Timp raspuns la client	Imediat (sau timp de fetch update)	Timp fetch update

In tabel este prezentata o comparatie intre protocoale push-based si pull-based in cazul sistemelor cu mai multi clienti si un singur server.

Protocoale de actualizare hibride

Contractul este o promisiune ca serverul va transmite actualizari la client pentru un timp specificat. Acest timp poate fi adaptat dinamic in functie de criteriile de contract (lease). Timpul poate fi calculat pe baza vechimii datelor astfel incat se incheie contracte de durata mai mare pentru elemente care au sansa de a ramane nemodificate. O alta posibilitate este calculul bazat pe frecventa de innoire astfel incat se incheie contracte de durata mai mare cu clientii ale caror cache-uri trebuie innoite frecvent. In fine, stabilirea timpului poate tine cont de incarcarea serverului. In aceasta varianta, serverul scade timpul de expirare al noilor contracte daca este supraincarcat (putand scadea numarul de clienti daca nu se reinnoiesc contractele).

Protocoale epidemice

Rolul protocoalelor epidemice este propagarea actualizarilor la replici, folosind un numar minim de mesaje. Ele se bazaza pe teoria epidemiilor care se ocupa de raspnadirea infectiilor. Abordarea folosita este una “anti-entropie”. Astfel, serverul P alege alt server Q la intamplare si schimba cu el actualizarile. Sunt posibile trei cazuri:

- ii transmite actualizarile
- preia de la el actualizarile
- face schimib de actualizari.

Sunt considerate mai multe categorii de servere:

Infectios – detine un update pe care vrea sa-l propage
Susceptibil – inca ne-actualizat
Eliminat – detine un update dar nu vrea sa-l propage.

De asemenea sunt mai multe abordari. Varianta "push based" este mai buna pentru putini infectiosi. In cazul multor infectiosi, probabilitatea de a selecta un susceptibil este mai mica. Pentru multi infectiosi este mai potrivita varianta "pull based".
O varianta speciala este raspandirea zvonurilor (gossiping) in care un server proaspat actualizat incearca sa propage la randul sau actualizarile. Interesul serverului de a propaga actualizarile scade pe masura ce intalneste servere actualizate deja. Protocoalele epidemice nu garanteaza actualizarea tuturor replicilor.

Protocoale de consistenta

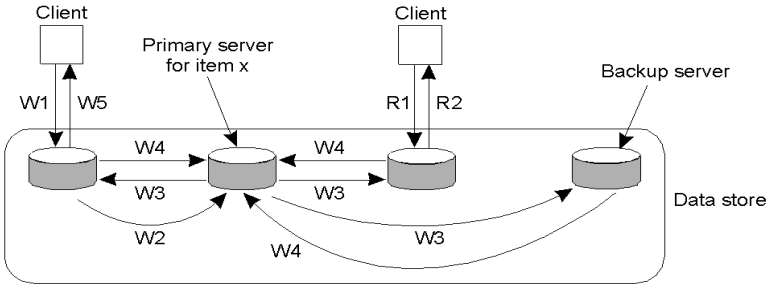
Un protocol de consistenta descrie o implementare a unui model de consistenta.
Clasificarea lor este urmatoarea:

Protocoale bazate pe o copie primara
scriere la distanta (Remote-write)

scriere locala (Local-write)
Protocoale cu scriere replicata
replicare activa
bazate pe cvorum
Protocoale de coerenta a cache-urilor.

Protocoale Remote-Write

In figura urmatoare este prezentat principiul protocolului primary-backup, la care scrierea este blocanta. O operatie de citire se face pe copia locala clientului. O cerere de scriere este transmisa copiei primare care face modifiarea si spune celorlalte copii sa faca actualizarea. Dupa transmiterea confirmarii modificarilor la serverul primar, executia scrierii se termina.



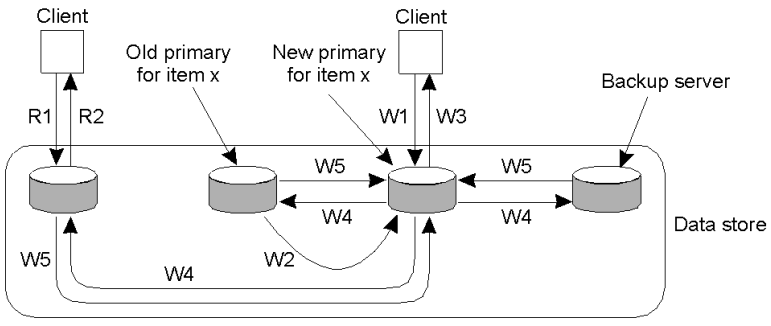
W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

Protocolul implementeaza consistenta secventiala (copia primara poate ordona toate operatiile de scriere primite). Pentru eficienta se poate face si scriere non-blocanta (se face actualizarea locala, apoi celelalte).

Protocoale Local-Write

O varianta este protocolul "primary-backup" in care copia primara migreaza la procesul care vrea sa faca actualizarea. Ordinea operatiilor este specificata in figura urmatoare.



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

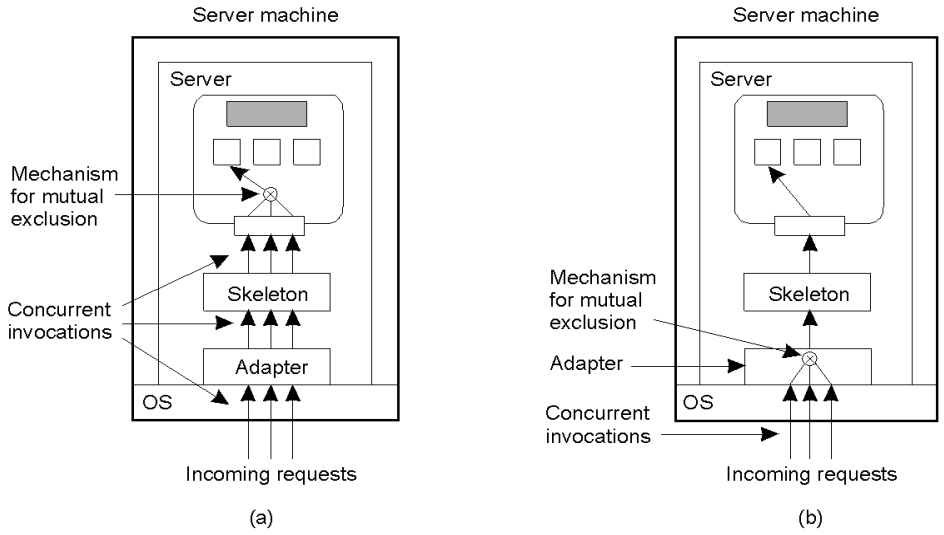
Replicare activa

In replicarea activa, fiecare replica are asociat un proces care executa operatiile de actualizare. O problema a replicarii active este ca operatiile trebuie executate in aceeasi ordine in toate replicile. Aceasta poate fi realizata prin folosirea unui mecanism de multicast total ordonat. In cazul replicarii obiectelor, de care ne ocupam in continuare, doar acest mecanism nu este suficient.

Pentru obiecte, consistenta la intrare este cea mai potrivita. Obiectele grupeaza in mod natural datele si operatiile, astfel ca sincronizarea prin lock a executiei metodelor serializeaza accesul si asigura consistenta datelor. La implementare trebuie rezolvate doua probleme:

– Prevenirea executiei concurente a invocarilor aceluiasi obiect (folosind lock-uri locale obiectului) – cu alte cuvinte, metodele aceluiasi obiect se executa in regim exclusiv.
– Pastrarea aceleiasi ordini a invocarilor la toate replicile.

Un server de obiecte (object server) nu furnizeaza el insusi servicii, ci prin obiectele care-i apartin. Serverul furnizeaza doar mijloacele de invocare a obiectelor locale pe baza cererilor clientilor. Componenta care sta la baza acestor mijloace este **adaptorul** (vezi figura urmatoare), care are cunostinta despre obiectele serverului si mijloceste tratarea invocarilor clientilor de catre obiectele corespunzatoare. Adaptoarele functioneaza in conformitate cu diverse **politici**. Astfel, pentru threaduri, politica cea mai simpla este de a avea un singur thread pentru un server. O alternativa este cea a mai multor threaduri pentru un server, cate unul pentru fiecare din obiectele sale. Este posibil sa se foloseasca un thread pentru fiecare invocare. In fine, este posibil ca threadul sa fie creat la cerere sau serverul sa aiba un grup cu un numar fix de threaduri create si alocate diferitelor invocari.

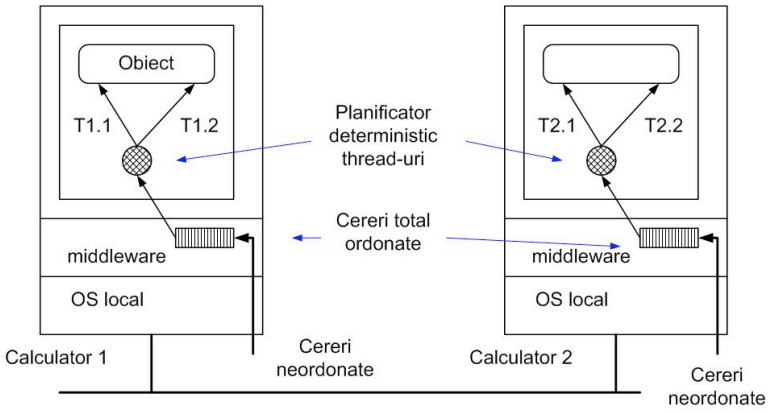


Pentru tratarea invocarilor concurente ale aceluiasi obiect se folosesc doua solutii. In prima, obiectul insusi poate trata invocarile concurente. De exemlu, metodele obiectului sunt sincronizate (synchronized) deci pot fi executate in regim de excludere mutuala (atomic). In cazul unor invocari simultane de la diversi clienti, vom avea mai multe threaduri dar unul va putea fi executat in timp ce celelalte vor fi blocate. In cea de a doua solutie, adaptorul asigura un singur thread per obiect asigurand executia exclusiva a

invocarilor si protejarea obiectului (care se presupune ca nu are mecanisme de sincronizare).

Pentru **asigurarea ordinii**, o solutie posibila este folosirea unei scheme primary-based la nivel de aplicatie. Schema permite serializarea cererilor dar cere implementarea in aplicatie a schemei primary-based, deci un efort suplimentar al dezvoltatorului de aplicatii care este implicat astfel in probleme din afara aplicatiei sale.

A doua solutie este implementarea in middelware a unui multicast total ordonat pentru invocari. In plus, trebuie asigurat si la nivelul executie threadurilor se trateaza invocarile pentru diferitele replici in ordinea corecta.



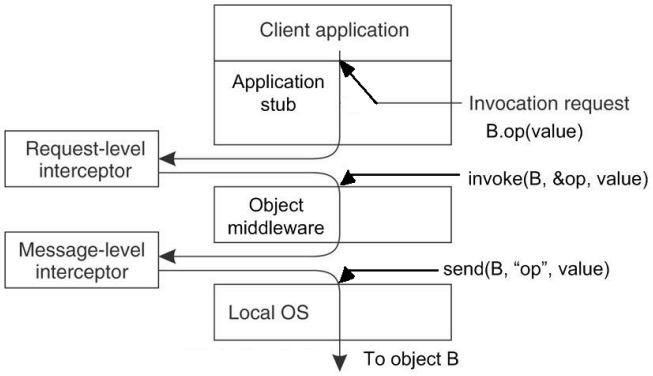
Problema ordonarii threadurilor apare datorita modului de functionare a serverelor multithread care:

- preiau o cerere
- o paseaza unui thread disponibil
- trec la urmatoarea cerere.

Deoarece planificatorul de threaduri aloca procesorul threadurilor executabile in functie de conditiile locale, ordinea threadurilor nu este aceeasi in toate replicile.
Solutionarea s-ar putea face printr-un planificator determinist, care are la baza o varianta de replica primara: copia primara determina ordinea threadurilor, o comunica celorlalte replici urmand ca acestea sa urmeze aceeasi ordine. Solutia presupune frecvente comunicari intre replici ceeace face procesul ineficient.

Separarea replicare de functionalitate obiecte

O alta problema este separarea replicarii de functionalitatea obiectelor astfel incat solutiile sa fie dezvoltate independent una de alta. Rezolvarea este bazata pe mecanismul interceptorilor, care sunt constructii software ce intrerup fluxul de control normal si permit intercalarea in fluxul de executie a altor functii.



In figura este ilustrata inserarea interceptorilor in invocarea unor metode la distanta. Clientului i se ofera o interfata identica cu cea a obiectului B de la distanta. Apelul B.op(value) este transformat intr-o invocare generica de metoda invoke(B, &op, value), care evidentiaza obiectul distribuit B, metoda invocata &op si valoarea parametrului operatiei. La randul sau, aceasta forma generica este transformata intr-un mesaj transmis prin retea.

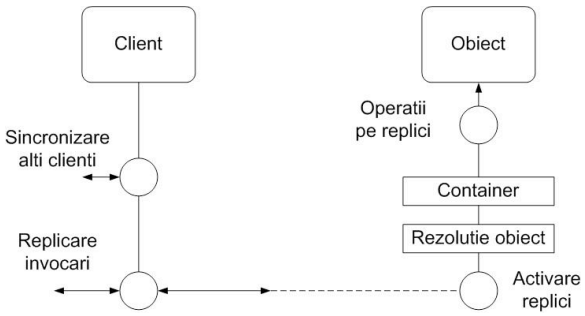
Daca **B este replicat** atunci se pot adauga doi interceptori:
Request-level interceptor – multiplica invocarea pentru fiecare replica
Message level interceptor – faciliteaza transmiterea; de exemplu, daca **value** este un tablou de mari dimensiuni acest interceptor poate fragmenta mesajul.

Ref.: Java IDL and Java RMI-IIOP Technologies: Using Portable Interceptors (PI)
http://java.sun.com/j2se/1.4.2/docs/guide/idl/PI.html

Un cadru complet de replicare

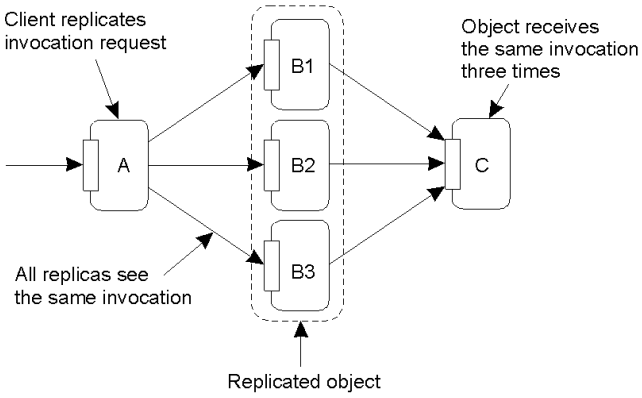
Ideea este de a insera interceptori in diferite puncte ale traseului invocarilor de obiecte (vezi figura urmatoare). La client se insereaza:
- un interceptor pentru sincronizare cu alti clienti, cand clientul este replicat;
- un interceptor pentru replicarea invocarii, cand aceasta trebuie transmisa mai multor replici obiect si pentru tratarea erorilor daca serverul un poate fi atins.

La server, se insereaza un interceptor splitat in doua parti:
- o parte care identifica obiectele server vizate si face activarea replicilor, daca invocarea se adreseaza mai multor replici;
- o parte care permite algoritmului de replicare sa citeasca sau sa seteze valorile atribut ale obiectului replicat.



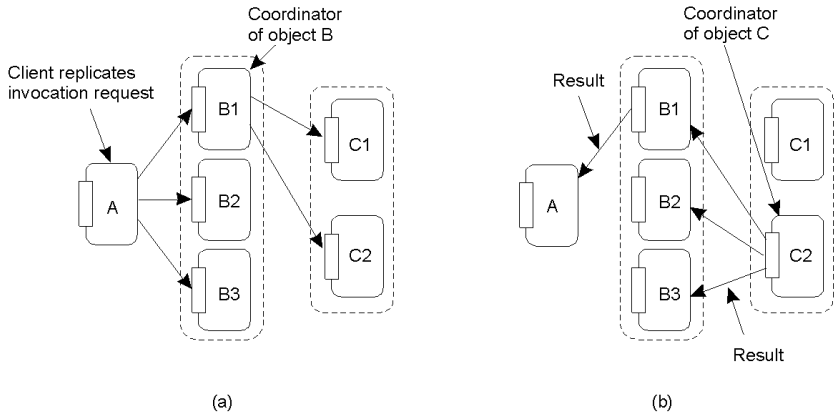
Invocari Replicate

O alta problema este cea a invocarilor replicate. In figura urmatoare, la C ajung mai multe invocari replicate, desi doar una are sens.



Solutie middleware: Communicatii constiente de replicare

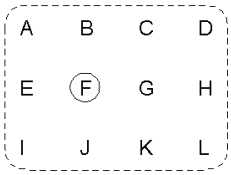
O solutie de tratare a invocarilor fara sens este bazata pe transmiterea multicast a invocarilor/raspunsurilor. Ea se bazeaza pe desemnare unui coordonator pentru fiecare grup de replici si pe asignarea de catre replici a aceluiasi identificator replicilor invocarilor. Doar coordonatorul transmite invocarea dar toate replicile primesc copii ale aceluiasi raspuns.



Alternativa ar fi identificarea si eliminarea invocarilor duplicate la nivelul obiectului.

Protocoale bazate pe cvorum

Protocoalele discutate pana acum sunt de tip ROWA (read one, write all): citirea se face pe replica locala, scrierea se face pe toate repllicile.



Aceasta varianta defavorizeaza operatiile de scriere care sunt mai greu de realizat. Daca operatiile de scriere prevaleaza atunci performanta sistemului sufera. O alternativa este oferita de protocoalele bazate pe cvorum, in care se folosesc doua cvorumuri, unul de citire si unul de scriere. Pentru a citi un item care are N replici, un client trebuie sa obtina acces la NR replici care constituie cvorumul de citire. Similar, pentru a scrie un item cu N replici, un client trebuie sa obtina acces la NW replici care constituie cvorumul de scriere. In cazul ROWA, avem NR=1 si NW=N.

Pentru ca operatiile sa se deruleze corect, NR si NW trebuie sa respecte doua conditii, prezentate in continuare.

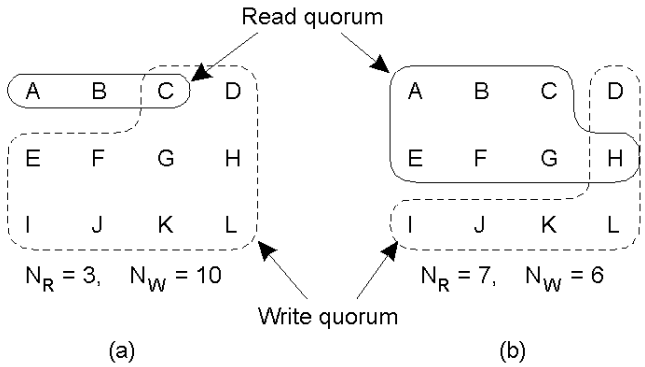
(a) Alegere corecta a seturilor de citire si scriere

NR + NW > N

Exista un numar de versiune pentru fiecare resursa. La o scriere, toate replicile scrise capata acelasi numar de versiune. Oricare ar fi alegerea cititorului, in cvorumul de citire intra cel putin o resursa care are ultima versiune (deoarece fiecare actualizare afecteaza NW replici).

(b) Alegere ce poate conduce la conflicte write-write

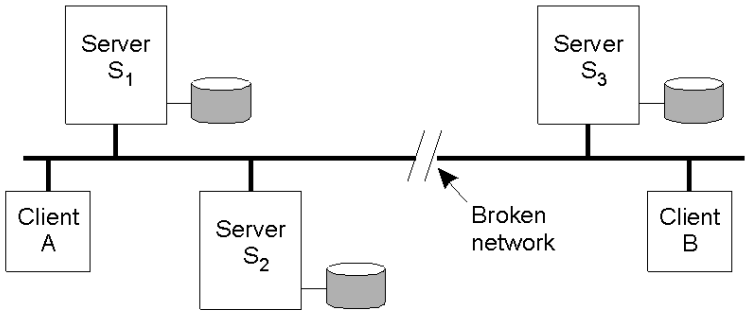
Sa consideram o situatie in care NW include cel mult jumatate din replici. Pentru situatia din figura urmatoare, este posibil ca un proces sa aleaga setul de scriere {A,B,C,E,F,G} iar altul {D,H,I,J,K,L}. Fiecare poate scrie pe replicile propriului set dand acelasi numar de versiune noilor valori. Obtinem astfel replici care au acelasi numar de versiune si valori diferite.



Pentru a evita aceasta situatie se impune ca **NW > N/2**

Aplicatie: replicarea serverelor in CODA

In CODA, unitatea de replicare este volumul. VSG (Volume Storage Group) este un grup de servere care au copia volumului respectiv, in timp ce AVSG (Accessible Volume Storage Group) reprezinta serverele la care un client are acces. In figura urmatoare, AVSG este {S1, S2} pentru clientul A si (S3) pentru clientul B.

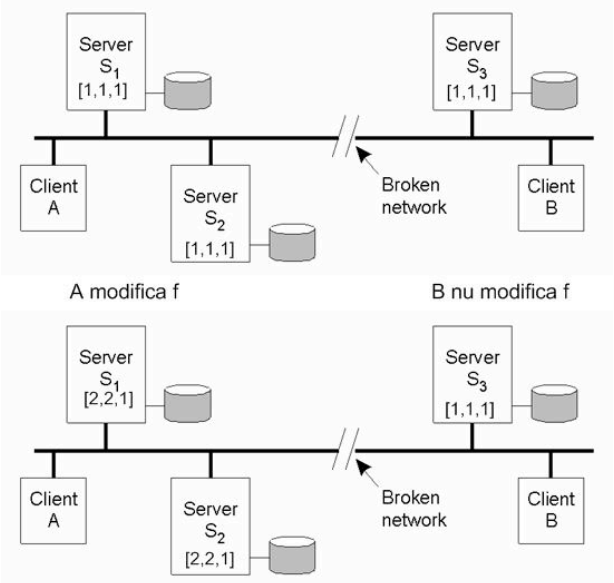


CODA foloseste ROWA pentru consistenta.Astfel, un utilizator poate deschide un fisier pe un membru AVSG, lucreaza pe el, iar la inchidere scrie toate replicile din AVSG.
Una din probleme este tratarea defectelor care partitioneaza reteaua. CODA adopta o varianta optimista de tratare a acestor situatii. Astfel, fiecare client lucreaza pe AVSG-ul sau si incearca sa trateze inconsistentele rezultate. Pentru detectia inconsistentelor, CODA pastreaza pentru fiecare fisier f si server k un vector de versiune (Coda Version Vector) $CVV_k(f)$ astfel ca:

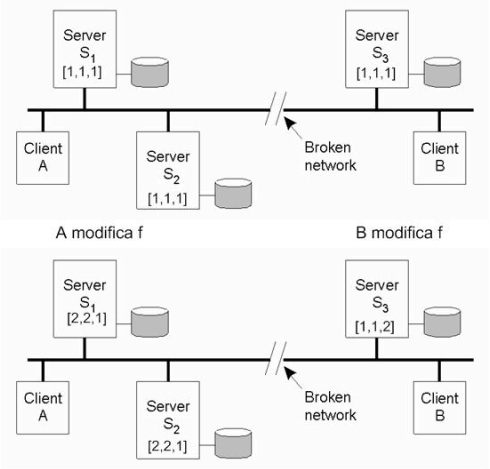
$CVV_k(f)[i]$ este numarul, pastrat de k, al versiunii curente a lui f pe serverul Si.

Pentru rezolvarea defectului de partitionare a retelei, se compara CVV-urile si se detecteaza conflictele. Sunt posibile mai multe situatii.

Fara conflict. Este situatia in care un acelasi fisier nu este modificat de utilizatori din partitii diferite. Un exemplu este ilustrat in figura urmatoare. Dupa refacerea retelei, modificarile sunt acceptate de S3 lucru reflectat de CVV-ul sau.



Conflict. Un fisier este modificat de utilizatori din fiecare partitie (ca in figura urmatoare).
Rezolvarea este dependenta de aplicatie, fiind posibila rezolvarea automata, dar de regula este necesara o solutie manuala.



Replicare (caching) in Web

Web foloseste mai multe tipuri de caching:
- la client – cache privat
- la proxy, server – cache-uri partajate.

Controlul caching, introdus in HTTP/1.1, se face de catre server prin antet Cache-Control cu valorile:
- public - nici o restrictie pentru caching
- private – nu in shared caches
- no-cache – nici in browser, nici in proxy.

Exemplu

```
HTTP/1.1 200 OK
Date: Mon, 05 Feb 2005 04:33:19 GMT
Server: Apache/1.2.5
Last-Modified: Mon, 05 Feb 2005 04:30:28 GMT
Cache-Control: private
Content-Length: 2289
…
```

Consistenta cache-urilor asigura ca documentul din cache este acelasi cu cel din server. Sunt mai multe forme prin care HTTP asigura acest lucru.

Solutia 1: Folosind comanda HEAD: clientul transmite HEAD si primeste raspunsul in care gaseste si verifica antetul Last-Modified.

Cerere:

```
HEAD http://www.cs.pub.ro/~ionescu/ HTTP/1.1
Host: www.cs.pub.ro
```

User-Agent: Mozilla/4.75 [en] (WinNT; U)

Raspuns:
HTTP/1.1 200 OK
Date: Mon, 05 Feb 2005 04:33:19 GMT
Server: Apache/1.2.5
Last-Modified: Mon, 05 Feb 2005 04:30:19 GMT
Content-Length: 2234
Content-Type: text/html

Daca documentul este mai nou dacat copia din cache se transmite GET.

Solutia 2: Folosind comanda GET cu antet If-Modified-Since.

GET /~ionescu/ HTTP/1.1
Host: www.cs.pub.ro
If-Modified-Since: Mon, 04 Feb 2005 04:30:28 GMT

Serverul transmite raspuns ca pagina nu a fost modificata:

HTTP/1.1 304 Not Modified
Date: Mon, 05 Feb 2005 04:33:19 GMT
Server: Apache/1.2.5

sau noul continut al paginii:

HTTP/1.1 200 OK
Date: Mon, 05 Feb 2005 04:33:19 GMT
Server: Apache/1.2.5
Last-Modified: Mon, 05 Feb 2005 04:30:28 GMT
Content-Length: 2289
…

Solutia 3. Clientul nu contacteaza serverul pentru orice cerere. Raspunsul unui server poate include data expirarii, care este memorata de client:

HTTP/1.1 200 OK
Date: Mon, 05 Feb 2005 04:33:20 GMT
Content-Type: image/jpeg
Content-Length: 35782
Cache-Control: private
Expires: Tue, 06 Feb 2005 04:33:20 GMT
Last-Modified: Mon, 05 Feb 2005 04:33:18 GMT

Clientul verifica existenta paginii in cache, cu urmatoarele posibilitati:

1. Daca pagina nu exista atunci cere resursa neconditionat.

2. Daca exista si este ne-expirata – foloseste intrarea din cache.
3. Daca exista si este expirata - adauga la cerere antetul If-Modified-Since si in continuare,
4. daca serverul raspunde cu 304 Not Modified foloseste intrarea din cache, altfel foloseste pagina primita.

Protocoale de coerenta cache-urilor

Este un caz special de replicare controlata de client. Una din problemele de proiectare este "Cand sunt detectate inconsistentele? (coherence detection strategy)". Sunt mai multe posibilitati.
Detectia se poate face static, la compilare. In principiu, se determina ce date pot conduce la inconsistente (sunt cache-uite) si se insereaza cod pentru ocolirea inconsistentei.
Detectia se poate face si dinamic, la executie. Se pot folosi trei politici:
- La accesarea unei date din cache se face mai intai verificarea consistentei.
- Lasa tranzactia sa se faca in paralel cu verificarea consistentei.
- Verifica consistenta datelor cand tranzactia se comite.

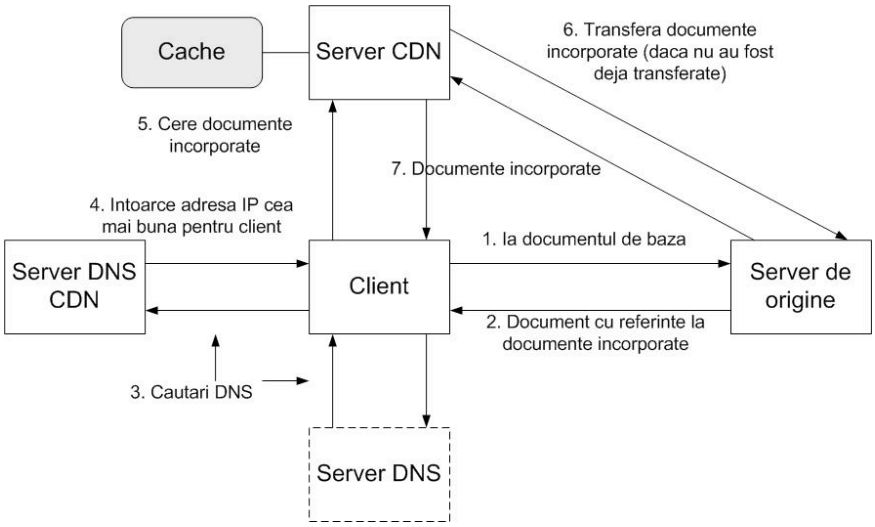
A doua intrebare este "Cum sunt cache-urile pastrate consistente?" (coherence enforcement strategy) Sunt mai multe politici. Una este de a nu permite cache-ul datelor partajate. O alta lasa serverul sa transmita invalidarea sau propagarea actualizarii.
In fine, o ultima intrebare este "Ce se intampla cand un proces modifica datele din cache?" Sunt doua solutii.
- Serverul modifica si propaga actualizarile la cache-uri (read-only)
- Clientul capata drept de acces exclusiv, modifica si paseaza actualizarile la servere (write-through caches).

Alte solutii de replicare – CDN – Content Delivery Network

CDN porneste de la presupunerea ca o pagina include multe alte documente (imagini, video etc.) care nu se schimba in timp. Aceste documente incorporate sunt replicate in servere Akamai CDN si livrate mai eficient clientilor.

Pentru aceste documente, in documentul de baza se foloseste un **URL modificat** care contine:
1. un **virtual ghost** – o referinta la documentul incorporat plus numele unui server DNS CDN
2. numele serverului de origine (care furnizeaza referintele la documentele incorporate).

CDN se bazeaza pe colectie de servere distribuite in Internet, care folosesc documente replicate si le ofera clientilor in functie de caracteristicile acestora. Pentru o mai buna eficienta, sunt monitorizate cererile clientilor pentru documentele CDN si se migreaza copiile in regiunea clientilor cu cele mai multe cereri, daca este cazul.



Solutia **Akamai** este prezentata in figura. Functionarea corespunde urmatorului tipar.
(1,2) Un client descarca un **document de baza** din **serverul de origine**. Documentul poate include URL-ul modificat al unui document CDN.
(3,4) Clientul rezolva numele CDN DNS folosind un DNS obisnuit. Apoi face o cerere catre CDN DNS pentru a gasi adresa IP a unui server CDN. Pentru a gasi un server CDN apropiat, in Akamai, DNS CDN cauta intr-o baza de date care mentine o harta a Internet-ului si care, in functie de adresa IP a clientului, returneaza adresa IP a serverului CDN apropiat de client. Ca alternativa, se poate da aceeasi adresa IP mai multor servere CDN, urmand ca cel mai apropiat sa fie gasit prin politica de rutare "calea cea mai scurta".
(5) Folosind adresa IP, clientul face o cerere catre serverul CDN caruia ii paseaza URL-ul modificat. Uzual, serverul CDN raspunde trimitand documentul incorporat din propriul cache.
(6,7) Daca serverul CDN nu are inca documentul cerut, il descarca folosind adresa serverului de origine din URL-ul modificat.